Term Work Index Sheet

Prof. name: Mrs. Jayshreee Patade		Class/SEM : T.Y.BSC.CS / SEM V (2026-2027)	
Course code: Minor - Statistics		Subject name: Regression Analysis using R - Programming	
Sr. No	**Date**	**Topics**	**Signature**
1		**Fundamentals of R Programming and Data Handling** To practice basic R commands, create common data structures and apply basic statistical functions.	
2		**Data Import, Pre-processing and Manipulation using R** To import, inspect, clean and manipulate a dataset in R.	
3		**Data Visualization and Exploratory Data Analysis using R** To visualize data and perform exploratory analysis before fitting a regression model	
4		**Simple Linear Regression using R** To fit a simple linear regression model, interpret its coefficients and predict response values.	
5		**Evaluation and Statistical Testing of Simple Linear Regression** To fit a simple linear regression model, interpret its coefficients and predict response values.	
6		**Multiple Linear Regression and Variable Selection using R** To fit a multiple linear regression model, interpret coefficients and explore variable-selection strategies.	
7		**Regression Model Validation and Residual Diagnostics** To calculate residual measures and use diagnostic plots to assess regression assumptions.	
8		**Detection and Treatment of Regression Assumption Violations** To test for autocorrelation and heteroscedasticity and consider suitable remedial measures.	

9		**Advanced Regression Techniques using R** To fit polynomial regression and weighted least squares models and compare model fit.	
10		**Multicollinearity, Ridge Regression and Regression Case Study** To identify multicollinearity, fit ridge regression and apply a complete regression workflow to a real-world dataset.	

Term Work Certification

Certified that Mr/Ms _____ Class _____ Roll No. _____ Course _____ has completed the required number of Practicals / Term Work / Sessional in the _____ in the Department of _____ during the academic year _____ successfully.

Signature of Faculty
Date:

Signature of HOD
Date:

Signature of Principal
Date:

Experiment No.— 1: Fundamentals of R Programming and Data Handling

Aim: To practice basic R commands, create common data structures and apply basic statistical functions.

Learning Objective:

To understand the basic concepts of R programming and data handling.

The purpose of this practical is to introduce students to basic R commands, data structures, and statistical functions used for handling and summarizing data.

Theory: R is a programming environment for statistical computing. Variables store values; vectors contain elements of one basic type, matrices are two-dimensional homogeneous structures, and data frames can contain columns of different types. Statistical functions summarize data.

Procedure:

1. Open R or RStudio and enter the commands in the script/editor.

2. Create variables and the requested data structures.

3. Apply statistical functions and observe the results.

R Program:

```
x <- c(12, 15, 18, 20, 25)
mat <- matrix(1:9, nrow=3, ncol=3)
students <- data.frame(id=c(1,2,3), marks=c(78,85,69))
mean(x)
sum(x)
min(x)
max(x)
range(x)
var(x)
summary(x)
str(students)
```

Function Used / Meaning:

str() – Displays the internal structure of an R object.

summary() – Displays a statistical summary of the data.

var() – Calculates the variance.

range() – Returns the minimum and maximum values.

max() – Returns the largest value.

min() – Returns the smallest value.

sum() – Calculates the total of the values.

mean() – Calculates the arithmetic mean (average).

data.frame() – Creates a data frame for storing data in rows and columns.

matrix() – Creates a matrix with the specified number of rows and columns.

c() – Creates a vector by combining values.

Output:

```
> x <- c(12, 15, 18, 20, 25)
>
> mat <- matrix(1:9, nrow=3, ncol=3)
>
> students <- data.frame(
+   id = c(1, 2, 3),
+   marks = c(78, 85, 69)
+ )
> mean(x)
[1] 18
> sum(x)
[1] 90
> min(x)
[1] 12
> max(x)
[1] 25
> range(x)
[1] 12 25
> var(x)
[1] 24.5
> summary(x)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
     12      15      18      18      20      25
> str(students)
'data.frame':   3 obs. of  2 variables:
 $ id   : num  1 2 3
 $ marks: num  78 85 69
```

Conclusion: The basic R commands, data structures and statistical functions were successfully practiced. The student learned how to create and summarize data using R.

Course Outcome: After completing this practical, the student will be able to use basic R commands, create common data structures and apply statistical functions.

Viva Questions:

1. What is R?
2. What is a vector in R?
3. How is a data frame different from a matrix?
4. What does mean() calculate?
5. What is the use of summary()?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 2: Data Import, Pre-processing and Manipulation using R

Aim: To import, inspect, clean and manipulate a dataset in R.

Learning Objective:

To learn how to import, inspect, clean, and manipulate data using R.

The purpose of this practical is to develop skills in reading datasets, identifying missing values, changing data types, creating variables, renaming or removing variables, and removing duplicate observations.

Theory: Data preprocessing prepares raw data for analysis. Common tasks include importing files, checking structure and summary statistics, handling missing values, converting data types, creating or renaming variables, and removing duplicate observations.

Procedure:

1. Set the working directory to the folder containing the dataset.

2. Import the CSV file and inspect its structure.

3. Handle missing values and duplicates; modify variables as required.

4. Export the cleaned dataset.

R Program:

```
#2A
emp <- data.frame(
  EmpID      = 1:6,
  Name       = c("Aarav", "Diya", "Kabir", "Meera", "Diya", "Rohan"),
  Age        = c(28, 34, NA, 45, 34, 30),
  Department = c("IT", "HR", "Finance", "IT", "HR", NA),
  Salary     = c(55000, 62000, 48000, NA, 62000, 51000)
)
emp
write.csv(emp, "employee_data.csv", row.names = FALSE)
emp_data <- read.csv("employee_data.csv", stringsAsFactors = FALSE)
emp_data
class(emp_data)
str(emp_data)
summary(emp_data)
#2b
emp$Age <- as.numeric(emp$Age)
emp$Department <- as.factor(emp$Department)
emp$Salary <- as.numeric(emp$Salary)
```

```
# Missing values
emp$Age[is.na(emp$Age)] <- 0
emp$Department[is.na(emp$Department)] <- "Unknown"
emp$Salary[is.na(emp$Salary)] <- 0

# Change variable types
emp$Age <- as.numeric(emp$Age)
emp$Department <- as.factor(emp$Department)

# Create a variable
emp$Bonus <- emp$Salary * 0.10

# Rename a variable
names(emp)[names(emp) == "Name"] <- "EmployeeName"

# Drop a variable
emp$EmpID <- NULL

# Remove duplicate observations
emp <- emp[!duplicated(emp), ]

emp
```

Important Note: Replace the file name and column names with those in your dataset.

Function Used / Meaning:

data.frame() – Creates a data frame from vectors of employee information.

write.csv() – Exports a data frame to a CSV file.

read.csv() – Imports data from a CSV file into R.

class() – Returns the class/type of an R object.

str() – Displays the structure and data types of an object.

summary() – Provides summary statistics for the dataset.

as.numeric() – Converts values to numeric type.

as.factor() – Converts a variable into a factor (categorical) type.

is.na() – Checks whether values are missing (NA).

names() – Gets or changes the names of variables/columns.

duplicated() – Identifies duplicate observations.

data.frame / $ – Creates data and accesses or modifies a particular column.

Output: 1)2A

```
> emp <- data.frame(
+   EmpID = 1:6,
+   Name = c("Aarav", "Diya", "Kabir", "Meera", "Diya", "Rohan"),
+   Age = c(28, 34, NA, 45, 34, 30),
+   Department = c("IT", "HR", "Finance", "IT", "HR", NA),
+   Salary = c(55000, 62000, 48000, NA, 62000, 51000)
+ )
> emp
  EmpID  Name Age Department Salary
1     1 Aarav  28         IT  55000
2     2  Diya  34         HR  62000
3     3 Kabir  NA    Finance  48000
4     4 Meera  45         IT     NA
5     5  Diya  34         HR  62000
6     6 Rohan  30       <NA>  51000
> write.csv(emp, "employee_data.csv", row.names = FALSE)
> emp_data <- read.csv("employee_data.csv", stringsAsFactors = FALSE)
> emp_data
  EmpID  Name Age Department Salary
1     1 Aarav  28         IT  55000
2     2  Diya  34         HR  62000
3     3 Kabir  NA    Finance  48000
4     4 Meera  45         IT     NA
5     5  Diya  34         HR  62000
6     6 Rohan  30       <NA>  51000
> class(emp_data)
[1] "data.frame"
> str(emp_data)
'data.frame':   6 obs. of  5 variables:
 $ EmpID     : int  1 2 3 4 5 6
 $ Name      : chr  "Aarav" "Diya" "Kabir" "Meera" ...
 $ Age       : int  28 34 NA 45 34 30
 $ Department: chr  "IT" "HR" "Finance" "IT" ...
 $ Salary    : int  55000 62000 48000 NA 62000 51000
> summary(emp_data)
     EmpID             Name                Age          Department          Salary
 Min.   :1.00   Length   :6   Min.   :28.0   Length   :6   Min.   :48000
 1st Qu.:2.25   N.unique :5   1st Qu.:30.0   N.unique :3   1st Qu.:51000
 Median :3.50   N.blank  :0   Median :34.0   N.blank  :0   Median :55000
 Mean   :3.50   Min.nchar:4   Mean   :34.2   Min.nchar:2   Mean   :55600
 3rd Qu.:4.75   Max.nchar:5   3rd Qu.:34.0   Max.nchar:7   3rd Qu.:62000
 Max.   :6.00                 Max.   :45.0   NAs      :1   Max.   :62000
                              NAs    :1                    NAs    :1
```

2)2B

```
> # Convert variable types
> emp$Age <- as.numeric(emp$Age)
> emp$Salary <- as.numeric(emp$Salary)
> # Handle missing values
> emp$Age[is.na(emp$Age)] <- 0
> emp$Department[is.na(emp$Department)] <- "Unknown"
> emp$Salary[is.na(emp$Salary)] <- 0
> # Convert Department to factor AFTER handling missing values
> emp$Department <- as.factor(emp$Department)
> # Create a variable
> emp$Bonus <- emp$Salary * 0.10
> # Rename a variable
> names(emp)[names(emp) == "Name"] <- "EmployeeName"
> # Drop a variable
> emp$EmpID <- NULL
> # Remove duplicate observations
> emp <- emp[!duplicated(emp), ]
> # Display final data
> emp
  EmployeeName Age Department Salary Bonus
1        Aarav  28         IT  55000  5500
2         Diya  34         HR  62000  6200
3        Kabir   0    Finance  48000  4800
4        Meera  45         IT      0     0
6        Rohan  30    Unknown  51000  5100
> |
```

Conclusion: The dataset was imported, inspected, cleaned and exported successfully. Missing values, data types and duplicate observations were handled using R.

Course Outcome: After completing this practical, the student will be able to import, inspect, clean, manipulate and export datasets using R.

Viva Questions:

1. What is a CSV file?
2. What does str() display?
3. How can missing values be identified?
4. What is the use of na.omit()?
5. Why are duplicate rows removed?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 3: Data Visualization and Exploratory Data Analysis using R

Aim: To visualize data and perform exploratory analysis before fitting a regression model.

Learning Objective:

To learn how to visualize data and perform exploratory data analysis using R.

The purpose of this practical is to help students understand data distributions, comparisons, category frequencies, proportions, and relationships between numerical variables through graphs.

Theory: Exploratory data analysis uses numerical summaries and graphs to understand distributions, unusual observations and relationships. Histograms show distributions, box plots summarize spread and possible outliers, and scatter plots display relationships between two numerical variables.

Procedure:

1. Load a dataset containing numerical and categorical variables.

2. Create histogram, box plot, bar chart and pie chart.

3. Create a scatter plot for a predictor and response variable.

R Program:

```
install.packages("ggplot2")    # Run once
library(ggplot2)
mtcars
head(mtcars)
#Histogram
p<- ggplot(mtcars, aes(x = mpg)) +
  geom_histogram(
    bins = 10,
    fill = "skyblue",
    color = "black"
  ) +
  labs(
    title = "Histogram of Miles Per Gallon",
    x = "Miles per Gallon",
    y = "Frequency"
  ) +
  theme_minimal()
ggsave("plot/histogram_mpg.png", plot = p)

#boxplot
b<- ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot(fill = "lightgreen", color = "black") +
  labs(
```

```
    title = "MPG by Number of Cylinders",
    x = "Cylinders",
    y = "Miles per Gallon"
  ) +
  theme_minimal()
ggsave("plot/box_mpg.png", plot = b)

#barDiagram
bar<-ggplot(mtcars, aes(x = factor(cyl))) +
  geom_bar(fill = "orange", color = "black") +
  labs(
    title = "Number of Cars by Cylinders",
    x = "Cylinders",
    y = "Number of Cars"
  ) +
  theme_minimal()
ggsave("plot/Bar_cyl.png", plot = b)

#barchar2
data <- data.frame(
  category = c("A", "B", "C", "D"),
  value = c(25, 40, 30, 20)
)

bar2<-ggplot(data, aes(x = category, y = value)) +
  geom_col(fill = "steelblue") +
  labs(title = "Category-wise Values", x = "Category", y = "Value") +
  theme_minimal()
ggsave("plot/bar2_mpg.png", plot = bar2)

#pie Chart
pie<- ggplot(data, aes(x = "", y = value, fill = category)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y") +
  labs(title = "Category Distribution") +
  theme_void()
ggsave("plot/pie1_mpg.png", plot = pie)

data$percentage <- round(
  data$value / sum(data$value) * 100, 1
)

pie2<-ggplot(data, aes(x = "", y = value, fill = category)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y") +
  geom_text(
    aes(label = paste0(percentage, "%")),
    position = position_stack(vjust = 0.5)
  ) +
```

```
  labs(title = "Category Distribution") +
  theme_void()
ggsave("plot/pie2_mpg.png", plot = pie2)

#Scatter Plot
s<- ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(color = "red", size = 3)  +
  labs(
    title = "Weight vs Miles per Gallon",
    x = "Weight",
    y = "Miles per Gallon"
  ) +
  theme_minimal()
ggsave("plot/Scatter_mpg.png", plot = s)
```

Function Used / Meaning:

install.packages() – Installs an R package from a repository; normally run once.

library() – Loads an installed package so its functions can be used.

ggplot() – Starts a ggplot2 graphics object using the specified data and mappings.

aes() – Defines how variables are mapped to visual properties such as x, y and fill.

geom_histogram() – Creates a histogram to display the distribution of a numerical variable.

geom_boxplot() – Creates a box plot showing median, spread and possible outliers.

geom_bar() – Creates a bar chart based on counts.

geom_col() – Creates a bar chart using supplied numerical values.

coord_polar() – Converts a bar chart into a circular/pie-style chart.

geom_point() – Creates points for a scatter plot.

geom_text() – Adds text labels to a graph.

labs() – Adds or changes the graph title and axis labels.

theme_minimal() – Applies a minimal graphical theme.

theme_void() – Removes most non-data elements from a graph.

ggsave() – Saves a ggplot graph as an image file.

factor() – Converts a variable into a categorical factor.

data.frame() – Creates a data frame for the category-wise example.

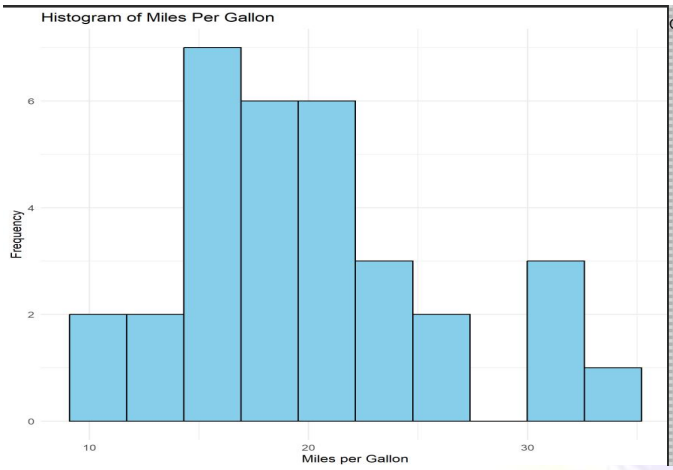
round() – Rounds numerical values to the specified number of decimal places.

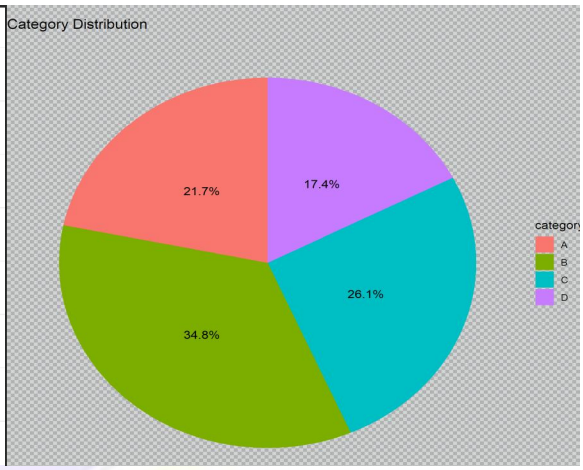
sum() – Calculates the total of numerical values.

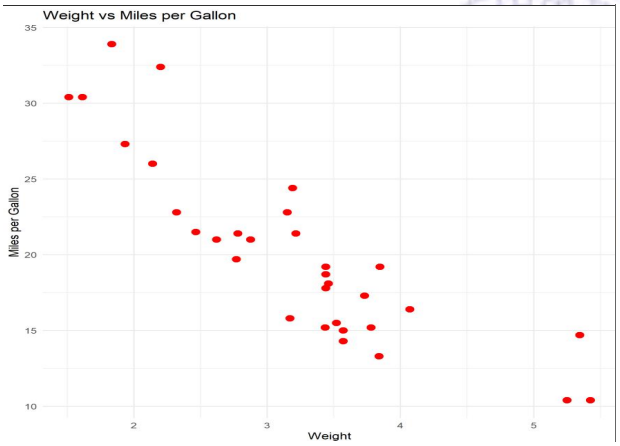
paste0() – Joins text without inserting separators.

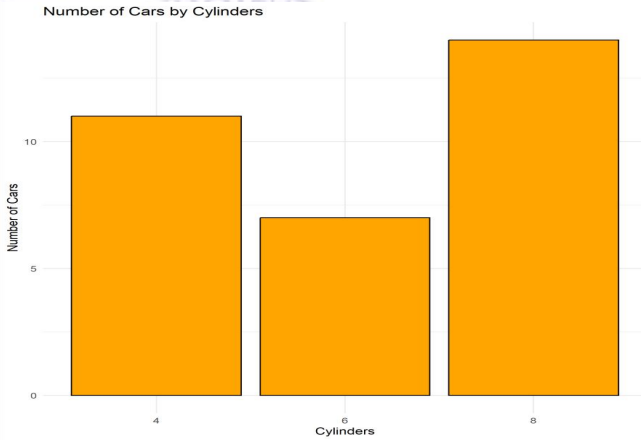
position_stack() – Controls positioning of stacked graphical elements.

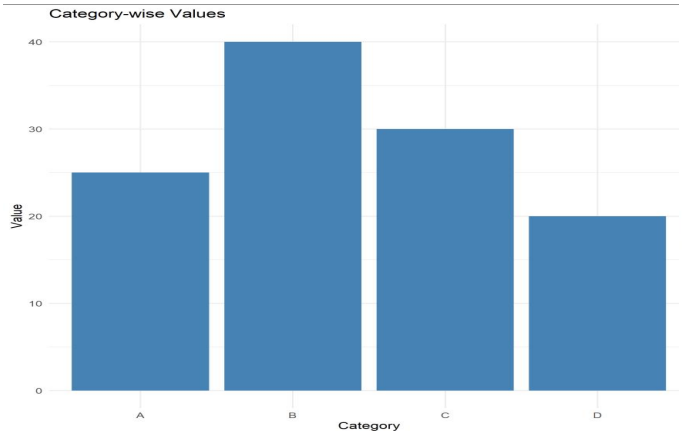
Output:

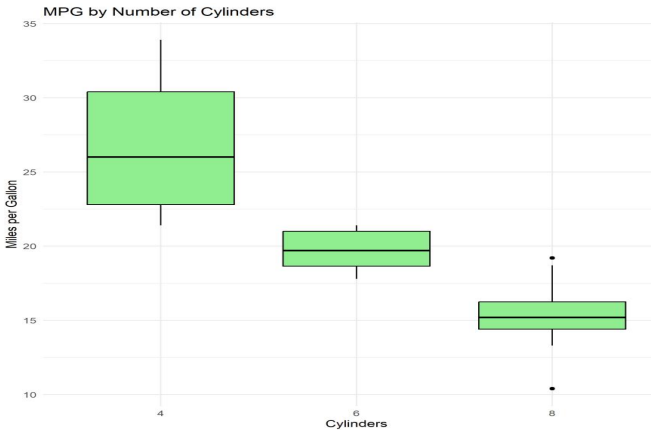












Conclusion: The dataset was explored using numerical summaries and graphical methods. Histograms, box plots, bar charts, pie charts and scatter plots helped understand distributions and relationships.

Course Outcome: After completing this practical, the student will be able to visualize data and perform basic exploratory data analysis using R.

Viva Questions:

1. What is exploratory data analysis?
2. What does a histogram show?
3. What is an outlier?
4. Why is a scatter plot useful in regression?
5. Name one R package used for visualization.

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 4: Simple Linear Regression using R

Aim: To fit a simple linear regression model, interpret its coefficients and predict response values.

Learning Objective:

To understand and implement simple linear regression using R.

The purpose of this practical is to fit a regression model, obtain the regression equation and coefficients, visualize the relationship between variables, and predict the response for given predictor values.

Theory: Simple linear regression models the relationship between one predictor x and a numerical response y. The fitted line is expressed as predicted y = intercept + slope × x. The lm() function fits the model and predict() generates fitted predictions.

Procedure:

1. Create or load a dataset with one predictor and one response variable.

2. Fit the model using lm().

3. Display coefficients and write the fitted equation.

4. Predict the response for selected predictor values.

R Program:

```
# SIMPLE LINEAR REGRESSION USING mtcars
#Part A
# Step 1: Load dataset
data(mtcars)

# Step 2: Display first few rows
head(mtcars)

# Step 3: Create linear regression model
model <- lm(mpg ~ hp, data = mtcars)

# Step 4: Display model summary
summary(model)

# Step 5: Display fitted equation
cat("Fitted Regression Equation:\n")
cat("mpg =", intercept, "+", slope, "* hp\n")

# Step 8: Plot data
plot(mtcars$hp, mtcars$mpg,
```

```
     main = "MPG vs Horsepower",
     xlab = "Horsepower (hp)",
     ylab = "Miles Per Gallon (mpg)",
     pch = 19)

# Step 9: Add regression line
abline(model)

#Part B
# Step 6: Store coefficients
coef(model)
# Step 7: Get regression coefficients
intercept <- coef(model)[1]
slope <- coef(model)[2]

# Predict MPG for hp = 150
new_car <- data.frame(hp = 150)
predict(model, newdata = new_car)

#Predict for multiple horsepower values
new_cars <- data.frame(hp = c(100, 150, 200, 250))
predicted_mpg <- predict(model, newdata = new_cars)

#tabluate output
result <- data.frame(
  Horsepower = new_cars$hp,
  Predicted_MPG = predicted_mpg
)
print(result)

#practical 5
# R-squared
# Step 1: Create linear regression model
model <- lm(mpg ~ hp, data = mtcars)
summary(model)

summary(model)$r.squared

# Adjusted R-squared
summary(model)$adj.r.squared

# t-test for individual regression coefficients
summary(model)$coefficients

# F-test for overall model significance
summary(model)$fstatistic
# 95% confidence intervals
confint(model)
```

```
# 99% confidence intervals (optional)
confint(model, level = 0.99)


#practical 6
```

Function Used / Meaning:

data() – Loads the built-in mtcars dataset.

head() – Displays the first few rows of a dataset.

lm() – Fits a linear regression model.

summary() – Displays detailed information about the fitted regression model.

cat() – Prints text and values together in the R console.

Execution Note: In the uploaded code, the fitted-equation cat() lines appear before intercept and slope are assigned. Run the coefficient section (Part B) before those cat() lines when executing the program sequentially.

coef() – Extracts the regression coefficients from the model.

plot() – Creates a graph of horsepower against miles per gallon.

abline() – Adds the fitted regression line to a plot.

data.frame() – Creates a data frame containing new predictor values for prediction.

predict() – Generates predicted response values for new observations.

Output:

```
> # Step 1: Load dataset
> data(mtcars)
> # Step 2: Display first few rows
> head(mtcars)
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```
> # Step 4: Display model summary
> summary(model)

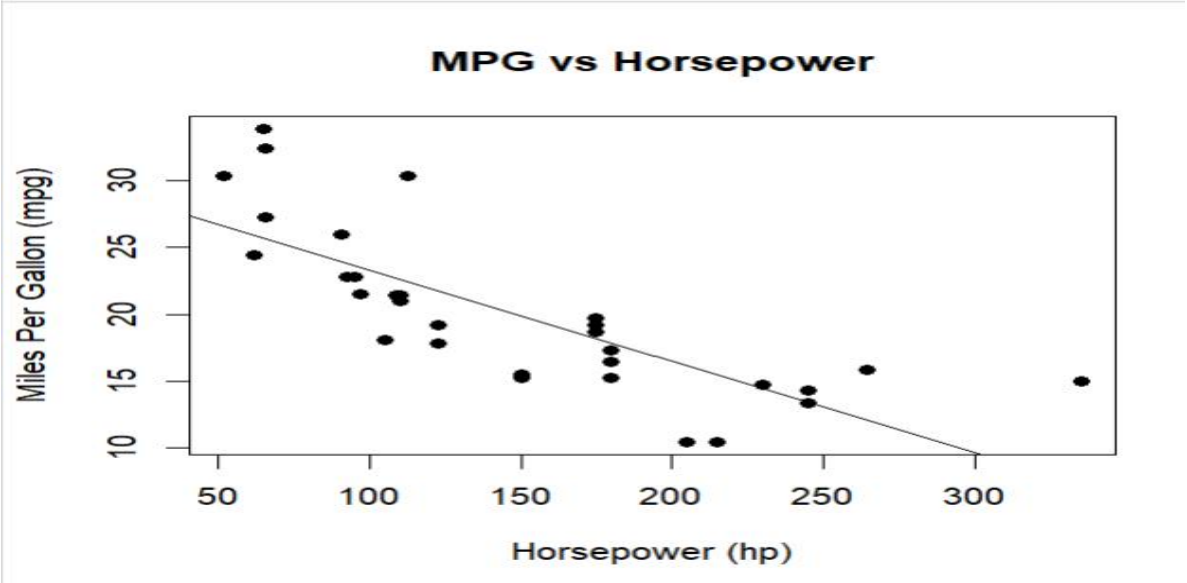
Call:
lm(formula = mpg ~ hp, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-5.7121 -2.1122 -0.8854  1.5819  8.2360

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 30.09886    1.63392  18.421  < 2e-16 ***
hp          -0.06823    0.01012  -6.742 1.79e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.863 on 30 degrees of freedom
Multiple R-squared:  0.6024,    Adjusted R-squared:  0.5892
F-statistic: 45.46 on 1 and 30 DF,  p-value: 1.788e-07

> # Step 5: Store coefficients
> intercept <- coef(model)[1]
> slope <- coef(model)[2]
> # Step 6: Display fitted equation
> cat("Fitted Regression Equation:\n")
Fitted Regression Equation:
> cat("mpg =", intercept, "+", slope, "* hp\n")
mpg = 30.09886 + -0.06822828 * hp
> # Step 7: Plot data
> plot(mtcars$hp, mtcars$mpg,
+      main = "MPG vs Horsepower",
+      xlab = "Horsepower (hp)",
+      ylab = "Miles Per Gallon (mpg)",
+      pch = 19)
> # Step 8: Add regression line
> abline(model)
> # Display regression coefficients
> coef(model)
```



```
> # Display regression coefficients
> coef(model)
(Intercept)          hp
30.09886054 -0.06822828
> # Predict MPG for hp = 150
> new_car <- data.frame(hp = 150)
> predict(model, newdata = new_car)
       1
19.86462
> # Predict for multiple horsepower values
> new_cars <- data.frame(hp = c(100, 150, 200, 250))
> predicted_mpg <- predict(model, newdata = new_cars)
> # Tabulate output
> result <- data.frame(
+   Horsepower = new_cars$hp,
+   Predicted_MPG = predicted_mpg
+ )
> print(result)
  Horsepower Predicted_MPG
1        100       23.27603
2        150       19.86462
3        200       16.45320
4        250       13.04179
```

Conclusion: A simple linear regression model was fitted successfully. The regression coefficients were obtained and response values were predicted for selected predictor values.

Course Outcome: After completing this practical, the student will be able to fit a simple linear regression model, interpret coefficients and generate predictions.

Viva Questions:

1. What is simple linear regression?
2. What is the response variable?
3. What does lm() do?
4. What is the meaning of the slope?
5. What is prediction in regression?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 5: Evaluation and Statistical Testing of Simple Linear Regression

Aim: To evaluate a simple regression model and perform statistical tests and confidence interval estimation.

Learning Objective:

To evaluate a regression model using statistical measures and tests.

The purpose of this practical is to understand R-squared, adjusted R-squared, coefficient t-tests, the overall F-test, and confidence intervals for assessing a regression model.

Theory: R-squared describes the proportion of response variation explained by the model. Adjusted R-squared accounts for the number of predictors. The coefficient t-tests assess individual coefficients, while the model F-test assesses overall regression significance. Confidence intervals provide a range of plausible parameter values.

Procedure:

1. Fit a simple linear regression model.

2. Read R-squared and adjusted R-squared from summary().

3. Examine coefficient t-tests and the overall F-test.

4. Calculate confidence intervals for model coefficients.

R Program:

```
#practical 5
# R-squared
# Step 1: Create linear regression model
model <- lm(mpg ~ hp, data = mtcars)
summary(model)

summary(model)$r.squared

# Adjusted R-squared
summary(model)$adj.r.squared

# t-test for individual regression coefficients
summary(model)$coefficients

# F-test for overall model significance
summary(model)$fstatistic
# 95% confidence intervals
confint(model)
```

```
# 99% confidence intervals (optional)
confint(model, level = 0.99)
```

Function Used / Meaning:

data() – Loads the built-in mtcars dataset.

lm() – Fits simple or multiple linear regression models.

summary() – Displays coefficients, R-squared, adjusted R-squared and significance information.

$r.squared – Extracts the R-squared value from a model summary.

$adj.r.squared – Extracts the adjusted R-squared value.

anova() – Compares nested regression models using analysis of variance.

step() – Performs stepwise model selection according to the selected direction.

par() – Sets graphical parameters such as the number of plots shown together.

plot() – Generates diagnostic plots for the selected regression model.

Output:

```
> # Step 1: Create linear regression model
> model <- lm(mpg ~ hp, data = mtcars)
> summary(model)

Call:
lm(formula = mpg ~ hp, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-5.7121 -2.1122 -0.8854  1.5819  8.2360

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 30.09886    1.63392  18.421  < 2e-16 ***
hp          -0.06823    0.01012  -6.742 1.79e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.863 on 30 degrees of freedom
Multiple R-squared:  0.6024,    Adjusted R-squared:  0.5892
F-statistic: 45.46 on 1 and 30 DF,  p-value: 1.788e-07
```

```
> summary(model)$r.squared
[1] 0.6024373
> # Adjusted R-squared
> summary(model)$adj.r.squared
[1] 0.5891853
> # t-test for individual regression coefficients
> summary(model)$coefficients
             Estimate Std. Error   t value     Pr(>|t|)
(Intercept) 30.09886054  1.6339210 18.421246 6.642736e-18
hp          -0.06822828  0.0101193 -6.742389 1.787835e-07
> # F-test for overall model significance
> summary(model)$fstatistic
  value   numdf   dendf
45.4598  1.0000 30.0000
> # 95% confidence intervals
> confint(model)
                 2.5 %     97.5 %
(Intercept) 26.76194879 33.4357723
hp          -0.08889465 -0.0475619
> # 99% confidence intervals (optional)
> confint(model, level = 0.99)
                 0.5 %      99.5 %
(Intercept) 25.60558503 34.59213605
hp          -0.09605632 -0.04040024
```

Conclusion: The simple regression model was evaluated using R-squared, adjusted R-squared, coefficient tests, the overall F-test and confidence intervals.

Course Outcome: After completing this practical, the student will be able to evaluate a regression model using R-squared, statistical tests and confidence intervals.

Viva Questions:

1. What does R-squared measure?
2. What is adjusted R-squared?
3. What is a t-test used for in regression?
4. What does the F-test assess?
5. What is a confidence interval?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 6: Multiple Linear Regression and Variable Selection using R

Aim: To fit a multiple linear regression model, interpret coefficients and explore variable-selection strategies.

Theory: Multiple linear regression uses two or more predictors to explain a numerical response. Each coefficient describes the expected change in the response for a one-unit increase in that predictor, holding other predictors constant. Model-building strategies should be guided by subject knowledge and diagnostics, not a single statistic alone.

Procedure:

1. Load a dataset with several possible predictors.

2. Fit a multiple regression model using lm().

3. Inspect coefficients, R-squared, adjusted R-squared and significance tests.

4. Compare a full model with a reduced model using a justified selection approach.

R Program:

```
#Multiple Regression
# Load data
data(mtcars)

# Model 1
model1 <- lm(mpg ~ wt, data = mtcars)

# Model 2
model2 <- lm(mpg ~ wt + hp, data = mtcars)

# Model 3
model3 <- lm(mpg ~ wt + hp + cyl, data = mtcars)

# Display results
summary(model1)
summary(model2)
summary(model3)

# Compare R-squared
summary(model1)$r.squared
summary(model1)$adj.r.squared

summary(model2)$r.squared
summary(model2)$adj.r.squared

summary(model3)$r.squared
summary(model3)$adj.r.squared
```

```
# Compare nested models
anova(model1, model2)
anova(model2, model3)

# Full model
full_model <- lm(
  mpg ~ wt + hp + disp + cyl + drat + qsec + vs + am + gear + carb,
  data = mtcars
)

# Variable selection
selected_model <- step(full_model, direction = "both")

summary(selected_model)

# Diagnostics
par(mfrow = c(2, 2))
plot(selected_model)
```

Function Used / Meaning:

lm() – Fits the regression model using the specified response and predictor.

residuals() – Returns residuals, calculated as observed values minus fitted values.

rstandard() – Calculates standardized residuals by scaling residuals using their estimated standard deviation.

rstudent() – Calculates studentized residuals using an error-variance estimate that excludes the observation being examined.

par() – Sets graphical parameters such as the plotting layout and margins.

plot() – Generates the selected regression diagnostic plots.

which = 1 – Selects the Residuals vs Fitted diagnostic plot.

which = 2 – Selects the Normal Q-Q diagnostic plot.

which = 3 – Selects the Scale-Location diagnostic plot.

which = 5 – Selects the Residuals vs Leverage diagnostic plot.

Output:

```
> # Load data
> data(mtcars)
> # Model 1
> model1 <- lm(mpg ~ wt, data = mtcars)
> # Model 2
> model2 <- lm(mpg ~ wt + hp, data = mtcars)
> # Model 3
> model3 <- lm(mpg ~ wt + hp + cyl, data = mtcars)
> # Display results
> summary(model1)

Call:
lm(formula = mpg ~ wt, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-4.5432 -2.3647 -0.1252  1.4096  6.8727

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  37.2851     1.8776  19.858  < 2e-16 ***
wt           -5.3445     0.5591  -9.559 1.29e-10 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared:  0.7528,    Adjusted R-squared:  0.7446
F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10
```

```
> summary(model2)

Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-3.941 -1.600 -0.182  1.050  5.854

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,    Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12
```

```
> summary(model3)

Call:
lm(formula = mpg ~ wt + hp + cyl, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9290 -1.5598 -0.5311  1.1850  5.8986

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 38.75179    1.78686  21.687  < 2e-16 ***
wt          -3.16697    0.74058  -4.276 0.000199 ***
hp          -0.01804    0.01188  -1.519 0.140015
cyl         -0.94162    0.55092  -1.709 0.098480 .
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

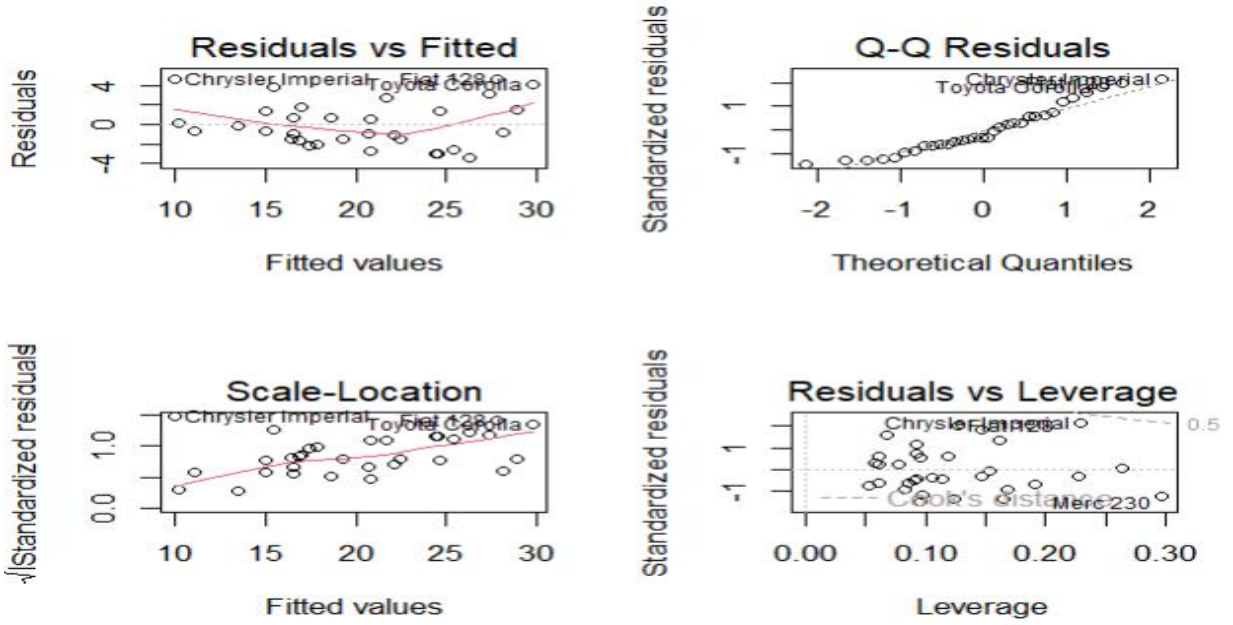
Residual standard error: 2.512 on 28 degrees of freedom
Multiple R-squared:  0.8431,    Adjusted R-squared:  0.8263
F-statistic: 50.17 on 3 and 28 DF,  p-value: 2.184e-11

> # Compare R-squared
> summary(model1)$r.squared
[1] 0.7528328
> summary(model1)$adj.r.squared
[1] 0.7445939
> summary(model2)$r.squared
[1] 0.8267855
> summary(model2)$adj.r.squared
[1] 0.8148396
> summary(model3)$r.squared
[1] 0.84315
> summary(model3)$adj.r.squared
[1] 0.8263446

> # Compare nested models
> anova(model1, model2)
Analysis of Variance Table

Model 1: mpg ~ wt
Model 2: mpg ~ wt + hp
  Res.Df    RSS Df Sum of Sq      F   Pr(>F)
1     30 278.32
2     29 195.05  1    83.274 12.381 0.001451 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
> anova(model2, model3)
Analysis of Variance Table

Model 1: mpg ~ wt + hp
Model 2: mpg ~ wt + hp + cyl
  Res.Df    RSS Df Sum of Sq      F  Pr(>F)
1     29 195.05
2     28 176.62  1    18.427 2.9213 0.09848 .
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
> # Full model
> full_model <- lm(
+   mpg ~ wt + hp + disp + cyl + drat + qsec + vs + am + gear + carb,
+   data = mtcars
+ )
```



Conclusion: A multiple linear regression model was fitted and compared with a reduced model. Model summaries, ANOVA and AIC were used to support model comparison.

Course Outcome: After completing this practical, the student will be able to fit multiple linear regression models and compare models using appropriate criteria.

Viva Questions:

1. What is multiple linear regression?
2. What is a predictor variable?
3. Why can adjusted R-squared be useful?
4. What is forward selection?
5. What does a regression coefficient represent?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 7: Regression Model Validation and Residual Diagnostics

Aim: To calculate residual measures and use diagnostic plots to assess regression assumptions.

Theory: Residuals are differences between observed and fitted response values. Diagnostic plots help identify nonlinearity, unequal variance, non-normality and influential observations. A model should be assessed using plots and context rather than relying only on summary statistics.

Procedure:

1. Fit a regression model.

2. Extract fitted values and residuals.

3. Calculate standardized and studentized residuals.

4. Generate and inspect the standard diagnostic plots.

R Program:

```
#Practical 7 - Regression Model Validation and Residual Diagnostics

model1 <- lm(mpg ~ wt, data = mtcars)

#Residual = Actual value - Predicted value
residuals(model1)

#Standardized residuals scale the residuals
#by their estimated standard deviation.
rstandard(model1)

#Studentized residuals are calculated by removing
#the observation being examined when estimating
#the error variance.
rstudent(model1)

par(mfrow = c(2, 2))
plot(model1, which = 1)  # Residuals vs Fitted

par(mfrow = c(1, 1))
par(mar = c(4, 4, 2, 2))
plot(model1, which = 2)  # Normal Q-Q

plot(model1, which = 3)  # Scale-Location
plot(model1, which = 5)  # Residuals vs Leverage

par(mfrow = c(1, 1))
```

```
##| Plot                       | Checks                          | Good indication
|
#  | -------------------------- | ------------------------------- | ------------------------
---------- |
#  | **Residuals vs Fitted**    | Linearity & constant variance | Random scatter around 0
|
#  | **Normal Q-Q**             | Normality of residuals         | Points close to straight
line       |
# | **Scale-Location**         | Constant variance              | Horizontal, evenly spread
pattern    |
# | **Residuals vs Leverage** | Influential observations       | No problematic high-
leverage points |
```

Function Used / Meaning:

plot() – Generates diagnostic plots for the regression model.

par() – Sets graphical parameters such as the plotting layout.

rstudent() – Calculates studentized residuals.

rstandard() – Calculates standardized residuals.

residuals() – Returns residuals, i.e. observed values minus fitted values.

fitted() – Returns fitted/predicted values from the model.

lm() – Fits the regression model.

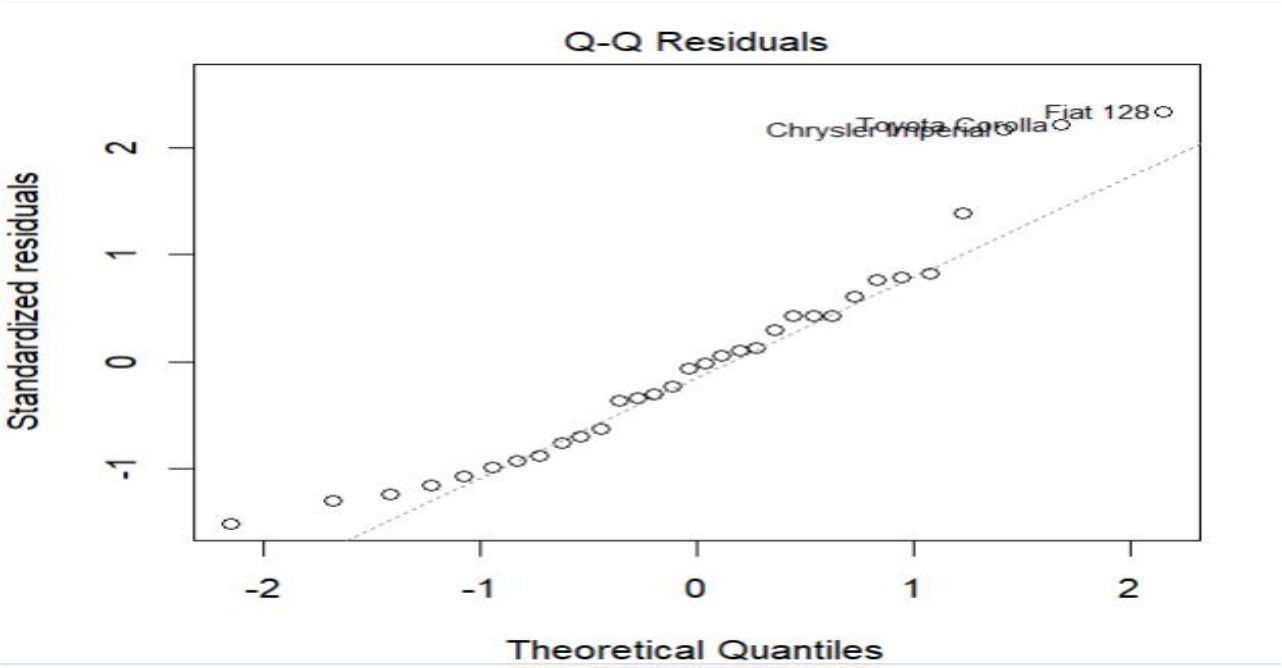
data() – Loads the mtcars dataset.

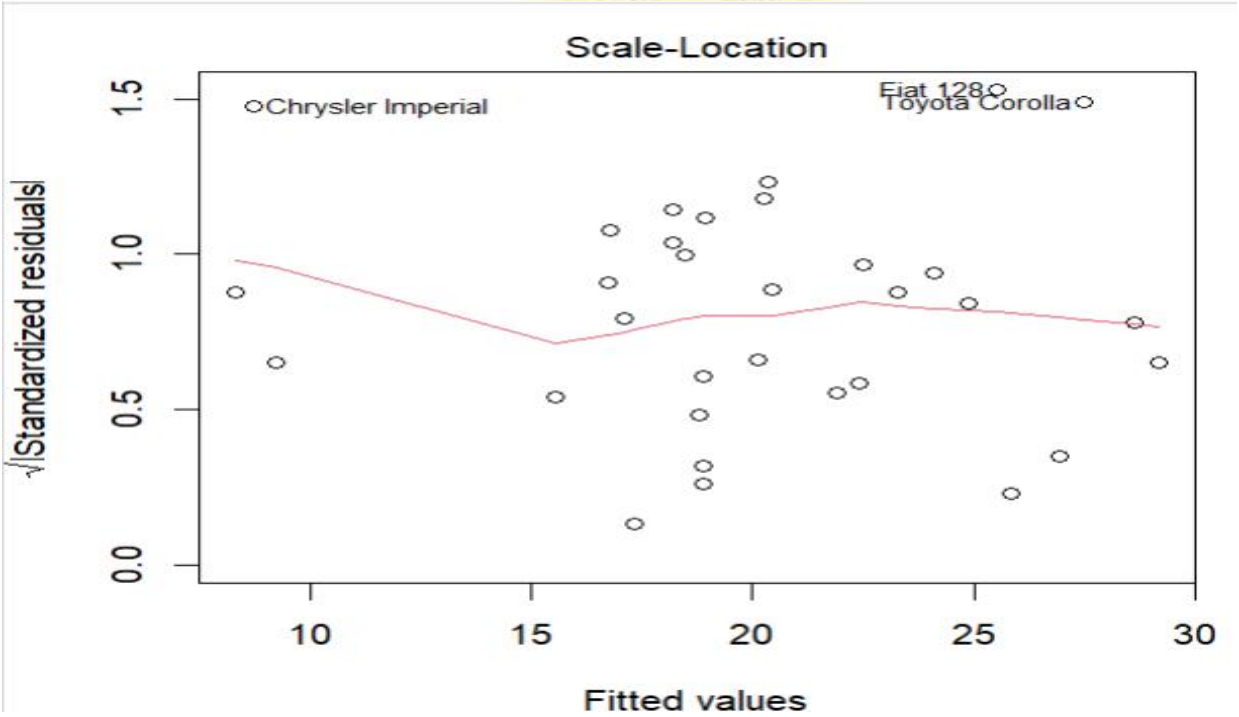
Output:

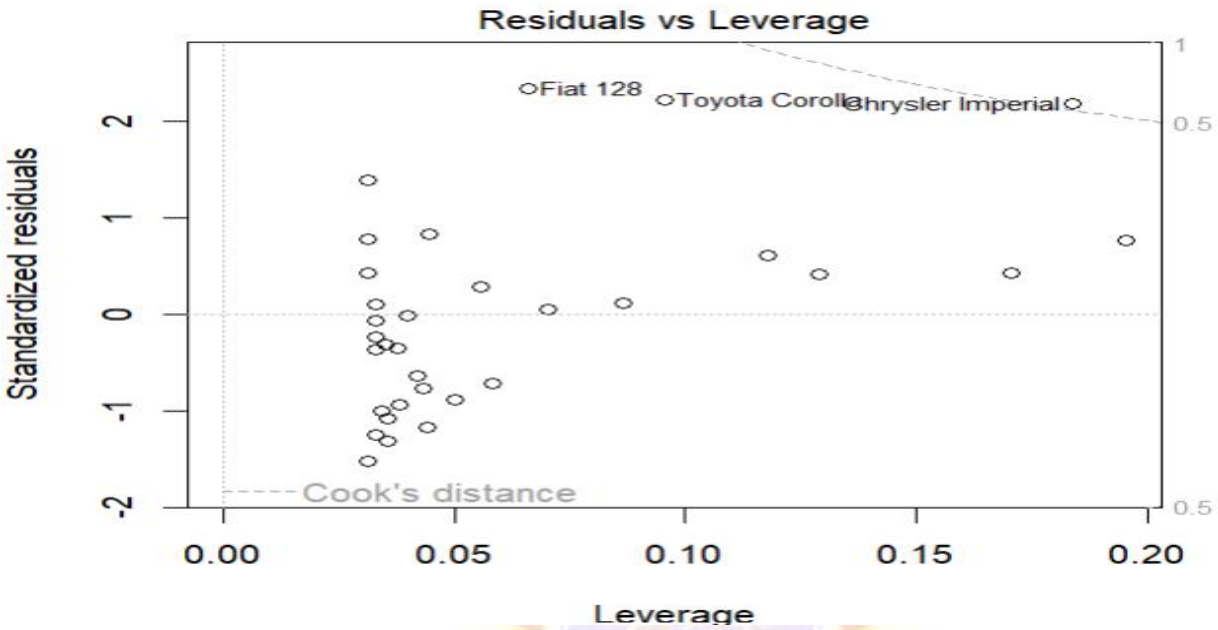
```
> model1 <- lm(mpg ~ wt, data = mtcars)
> #Residual = Actual value - Predicted value
> residuals(model1)
          Mazda RX4       Mazda RX4 Wag          Datsun 710      Hornet 4 Drive   Hornet Sportabout
         -2.2826106          -0.9197704          -2.0859521           1.2973499          -0.2001440
            Valiant          Duster 360           Merc 240D            Merc 230            Merc 280
         -0.6932545          -3.9053627           4.1637381           2.3499593           0.2998560
           Merc 280C          Merc 450SE          Merc 450SL         Merc 450SLC Cadillac Fleetwood
         -1.1001440           0.8668731          -0.0502472          -1.8830236           1.1733496
Lincoln Continental   Chrysler Imperial            Fiat 128          Honda Civic       Toyota Corolla
          2.1032876           5.9810744           6.8727113           1.7461954           6.4219792
      Toyota Corona    Dodge Challenger          AMC Javelin           Camaro Z28    Pontiac Firebird
         -2.6110037          -2.9725862          -3.7268663          -3.4623553           2.4643670
           Fiat X1-9       Porsche 914-2         Lotus Europa      Ford Pantera L         Ferrari Dino
           0.3564263           0.1520430           1.2010593          -4.5431513          -2.7809399
      Maserati Bora          Volvo 142E
         -3.2053627          -1.0274952
```

```
> #Standardized residuals scale the residuals
> #by their estimated standard deviation.
> rstandard(model1)
          Mazda RX4       Mazda RX4 Wag          Datsun 710      Hornet 4 Drive   Hornet Sportabout
         -0.76616765         -0.30743051         -0.70575249          0.43275114         -0.06681879
             Valiant          Duster 360           Merc 240D            Merc 230            Merc 280
         -0.23148309         -1.30552216          1.38889709          0.78392687          0.10010803
            Merc 280C           Merc 450SE          Merc 450SL         Merc 450SLC  Cadillac Fleetwood
         -0.36728706          0.29288651         -0.01683789         -0.63159969          0.42296071
Lincoln Continental   Chrysler Imperial             Fiat 128          Honda Civic       Toyota Corolla
          0.76979873          2.17353314          2.33490215          0.61035691          2.21708271
       Toyota Corona    Dodge Challenger          AMC Javelin          Camaro Z28    Pontiac Firebird
         -0.87964013         -0.99313634         -1.24418015         -1.16279098          0.82771968
           Fiat X1-9       Porsche 914-2         Lotus Europa       Ford Pantera L         Ferrari Dino
          0.12244407          0.05177187          0.42254270         -1.51549710         -0.93086929
       Maserati Bora           Volvo 142E
         -1.07151943         -0.34388215
> #Studentized residuals are calculated by removing
> #the observation being examined when estimating
> #the error variance.
> rstudent(model1)
          Mazda RX4       Mazda RX4 Wag          Datsun 710      Hornet 4 Drive   Hornet Sportabout
         -0.76076977         -0.30274050         -0.69972329          0.42681176         -0.06570059
             Valiant          Duster 360           Merc 240D            Merc 230            Merc 280
         -0.22779587         -1.32167204          1.41169344          0.77876838          0.09844187
            Merc 280C           Merc 450SE          Merc 450SL         Merc 450SLC  Cadillac Fleetwood
         -0.36192838          0.28837628         -0.01655496         -0.62515414          0.41709709
Lincoln Continental   Chrysler Imperial             Fiat 128          Honda Civic       Toyota Corolla
          0.76444774          2.32816206          2.53780106          0.60385911          2.38384376
       Toyota Corona    Dodge Challenger          AMC Javelin          Camaro Z28    Pontiac Firebird
         -0.87622897         -0.99290217         -1.25610443         -1.16991451          0.82326228
           Fiat X1-9       Porsche 914-2         Lotus Europa       Ford Pantera L         Ferrari Dino
          0.12041613          0.05090396          0.41668241         -1.55056010         -0.92873434
       Maserati Bora           Volvo 142E
         -1.07426703         -0.33877056

> par(mfrow = c(2, 2))
> plot(model1, which = 1)  # Residuals vs Fitted
> par(mfrow = c(1, 1))
> par(mar = c(4, 4, 2, 2))
> plot(model1, which = 2)  # Normal Q-Q
> plot(model1, which = 3)  # Scale-Location
> plot(model1, which = 5)  # Residuals vs Leverage
> par(mfrow = c(1, 1))
>
```


Q-Q Residuals
Standardized residuals
Theoretical Quantiles
Fiat 128
Toyota Corolla
Chrysler Imperial
-2
-1
0
1
2


Scale-Location
√|Standardized residuals|
Fitted values
Chrysler Imperial
Fiat 128
Toyota Corolla
0.0
0.5
1.0
1.5
10
15
20
25
30



Conclusion: Residual measures and standard diagnostic plots were generated successfully. They can be used to assess linearity, constant variance, normality and influential observations.

Course Outcome: After completing this practical, the student will be able to calculate residual measures and use diagnostic plots for regression model validation.

Viva Questions:

1. What is a residual?
2. What is a fitted value?
3. What does a residual plot help detect?
4. Name the four standard diagnostic plots.
5. What is normality of residuals?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 8: Detection and Treatment of Regression Assumption Violations

Aim: To test for autocorrelation and heteroscedasticity and consider suitable remedial measures.

Theory: Autocorrelation refers to dependence among errors, often relevant in ordered or time-series data. Heteroscedasticity means error variance is not constant. The Durbin-Watson test and Breusch-Pagan test are commonly used to investigate these issues. Test conclusions should be interpreted alongside plots and study design.

Procedure:

1. Install and load the lmtest package if it is not already installed.

2. Fit a regression model to suitable data.

3. Run the Durbin-Watson and Breusch-Pagan tests.

4. Interpret the test statistics and p-values; discuss an appropriate remedy if an assumption appears violated.

R Program:

```
# install.packages("lmtest")
library(lmtest)
data(mtcars)
model <- lm(mpg ~ wt + hp, data=mtcars)
dwtest(model)
bptest(model)
plot(fitted(model), residuals(model), xlab="Fitted Values", ylab="Residuals")
abline(h=0, lty=2)
```

Important Note: The Durbin-Watson test is most meaningful when the observation order has substantive meaning.

Function Used / Meaning:

abline() – Adds a straight reference line to a graph.

plot() – Creates a plot of fitted values against residuals.

bptest() – Performs the Breusch-Pagan test for heteroscedasticity.

dwtest() – Performs the Durbin-Watson test for autocorrelation.

lm() – Fits the regression model.

data() – Loads the mtcars dataset.

library() – Loads an installed R package for use.

Output:

```
> library(lmtest)
Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric

> # install.packages("lmtest")
> library(lmtest)
> data(mtcars)
> model <- lm(mpg ~ wt + hp, data = mtcars)
> dwtest(model)

        Durbin-Watson test

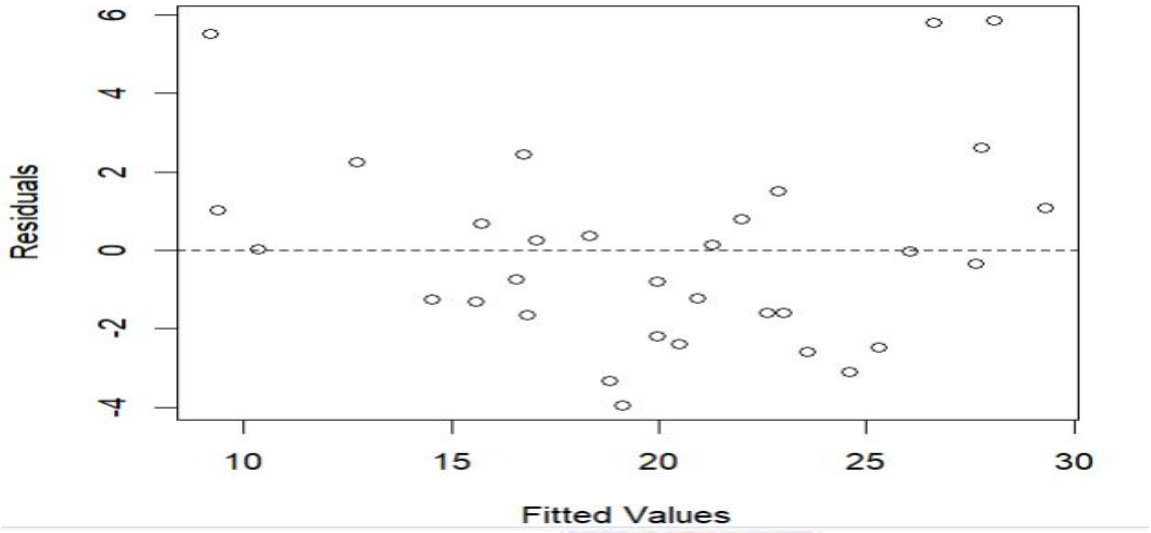
data:  model
DW = 1.3624, p-value = 0.02061
alternative hypothesis: true autocorrelation is greater than 0

> bptest(model)

        studentized Breusch-Pagan test

data:  model
BP = 0.88072, df = 2, p-value = 0.6438

> plot(fitted(model), residuals(model),
+      xlab = "Fitted Values",
+      ylab = "Residuals")
> abline(h = 0, lty = 2)
```



Conclusion: The Durbin-Watson and Breusch-Pagan tests were applied to investigate autocorrelation and heteroscedasticity, supported by residual plots.

Course Outcome: After completing this practical, the student will be able to identify autocorrelation and heteroscedasticity using statistical tests and residual plots.

Viva Questions:

1. What is autocorrelation?
2. What does the Durbin-Watson test examine?
3. What is heteroscedasticity?
4. What does bptest() test?
5. Name one remedy for non-constant variance.

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 9: Advanced Regression Techniques using R

Aim: To fit polynomial regression and weighted least squares models and compare model fit.

Theory: Polynomial regression adds powers of a predictor to represent curved relationships while remaining linear in the coefficients. Weighted least squares gives observations different weights, often to address a known or modelled variance pattern. Model comparisons should consider assumptions and validation as well as fit.

Procedure:

1. Plot the predictor-response relationship and consider whether curvature is plausible.

2. Fit linear and polynomial models.

3. Compare model summaries and residual plots.

4. Fit a weighted least squares model using justified weights.

R Program:

```
data(cars)
linear_model <- lm(dist ~ speed, data=cars)
poly_model <- lm(dist ~ speed + I(speed^2), data=cars)
summary(linear_model)
summary(poly_model)
anova(linear_model, poly_model)
plot(poly_model)
# Example weights for demonstration only; choose weights based on the data/model
wls_model <- lm(dist ~ speed, data=cars, weights=1/speed^2)
summary(wls_model)
```

Important Note: Weights must be selected based on a defensible variance model; the example is for demonstrating syntax.

Function Used / Meaning:

plot() – Creates diagnostic plots for the fitted model.

anova() – Compares the linear and polynomial models.

summary() – Displays regression model results.

I() – Forces an expression such as speed^2 to be interpreted as a mathematical term in the model formula.

lm() – Fits linear, polynomial or weighted least squares regression models.

data() – Loads the cars dataset.

Output:

```
> data(cars)
> # Linear Regression Model
> linear_model <- lm(dist ~ speed, data = cars)
> summary(linear_model)

Call:
lm(formula = dist ~ speed, data = cars)

Residuals:
    Min      1Q  Median      3Q     Max
-29.069  -9.525  -2.272   9.215  43.201

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -17.5791     6.7584  -2.601   0.0123 *
speed         3.9324     0.4155   9.464 1.49e-12 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 15.38 on 48 degrees of freedom
Multiple R-squared:  0.6511,    Adjusted R-squared:  0.6438
F-statistic: 89.57 on 1 and 48 DF,  p-value: 1.49e-12

> # Polynomial Regression Model
> poly_model <- lm(dist ~ speed + I(speed^2), data = cars)
> summary(poly_model)

Call:
lm(formula = dist ~ speed + I(speed^2), data = cars)

Residuals:
    Min      1Q  Median      3Q     Max
-28.720  -9.184  -3.188   4.628  45.152

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.47014   14.81716   0.167    0.868
speed        0.91329    2.03422   0.449    0.656
I(speed^2)   0.09996    0.06597   1.515    0.136
```

```
> # Compare Linear and Polynomial Models
> anova(linear_model, poly_model)
Analysis of Variance Table

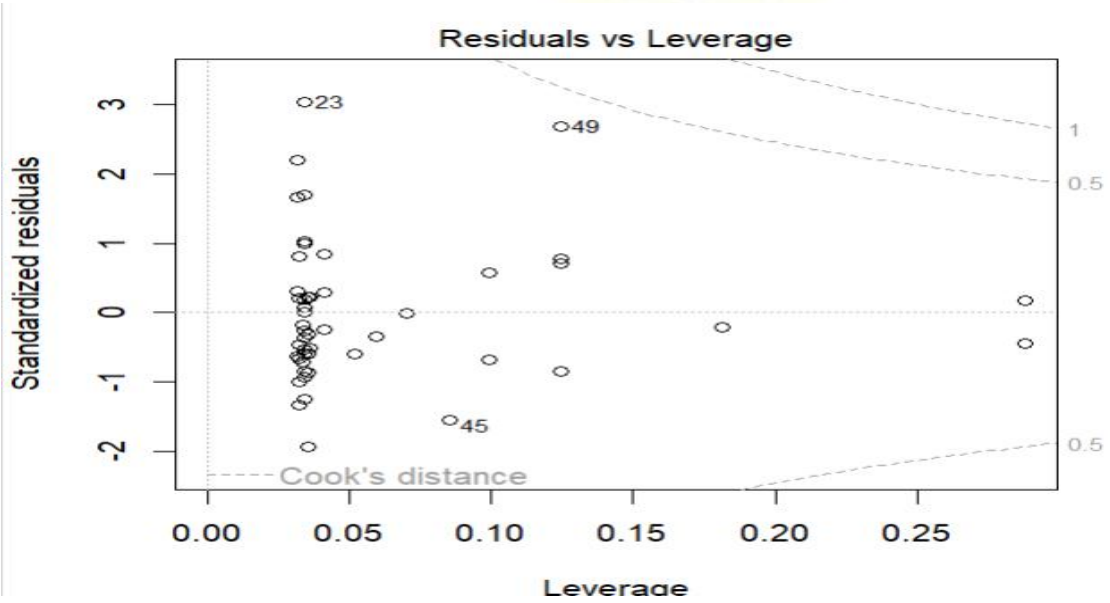
Model 1: dist ~ speed
Model 2: dist ~ speed + I(speed^2)
  Res.Df   RSS Df Sum of Sq     F Pr(>F)
1     48 11354
2     47 10825  1    528.81 2.296 0.1364
> # Plot Polynomial Regression Diagnostics
> plot(poly_model)
Hit <Return> to see next plot: # Weighted Least Squares Model
Hit <Return> to see next plot: # Example weights for demonstration only
Hit <Return> to see next plot: wls_model <- lm(dist ~ speed, data = cars,
Hit <Return> to see next plot:                   weights = 1/speed^2)
> summary(wls_model)

Call:
lm(formula = dist ~ speed, data = cars, weights = 1/speed^2)

Weighted Residuals:
    Min      1Q  Median      3Q     Max
-1.4324 -0.6295 -0.1453  0.6369  3.0270

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -9.5676     3.2842  -2.913  0.00542 **
speed         3.3706     0.2898  11.631 1.44e-15 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.9947 on 48 degrees of freedom
Multiple R-squared:  0.7381,    Adjusted R-squared:  0.7326
F-statistic: 135.3 on 1 and 48 DF,  p-value: 1.437e-15
```



Conclusion: Linear, polynomial and weighted regression models were fitted and compared. The practical demonstrated how model form and justified weights can be used when appropriate.

Course Outcome: After completing this practical, the student will be able to apply polynomial regression and weighted least squares and compare regression models.

Viva Questions:

1. What is polynomial regression?
2. Why include a squared predictor term?
3. What is weighted least squares?
4. When might a response transformation help?
5. Why compare regression models?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.— 10: Multicollinearity, Ridge Regression and Regression Case Study

Aim: To identify multicollinearity, fit ridge regression and apply a complete regression workflow to a real-world dataset.

Theory: Multicollinearity occurs when predictors are strongly related, which can make coefficient estimates unstable. Correlation matrices and variance inflation factors (VIF) help diagnose it. Ridge regression adds an L2 penalty to reduce coefficient variance. Model evaluation should include data inspection, fitting, diagnostics and validation.

Procedure:

1. Inspect the predictor correlations and calculate VIF values.

2. Fit an ordinary least squares model and note its coefficients.

3. Fit a ridge model using a package and select the penalty parameter using cross-validation.

4. Compare model performance and summarize the workflow.

R Program:

```
# PRACTICAL NO. 10
# Multicollinearity, Ridge Regression and Regression Case Study

# Install packages only if not already installed
# install.packages(c("car", "MASS", "glmnet"))

library(car)
library(MASS)
library(glmnet)

# Load dataset
data(mtcars)

# ---------------------------------
# PART A: MULTICOLLINEARITY
# ---------------------------------

# Fit multiple linear regression model
ols <- lm(mpg ~ wt + hp + disp + qsec, data = mtcars)

# Correlation matrix
cor(mtcars[, c("wt", "hp", "disp", "qsec")])

# Variance Inflation Factor
vif(ols)
```

```
# ---------------------------------
# PART B: RIDGE REGRESSION
# ---------------------------------

# Prepare predictor matrix and target variable
x <- model.matrix(mpg ~ wt + hp + disp + qsec,
                  data = mtcars)[, -1]
y <- mtcars$mpg

# Ridge regression with cross-validation
set.seed(123)
cv_ridge <- cv.glmnet(x, y, alpha = 0)

# Best regularization parameter
cv_ridge$lambda.min

# Ridge regression coefficients
coef(cv_ridge, s = "lambda.min")

# Predictions for first five observations
predict(cv_ridge, newx = x[1:5, , drop = FALSE],
        s = "lambda.min")

# ---------------------------------
# PART C: REGRESSION CASE STUDY
# ---------------------------------

# Split data into training and testing sets
set.seed(123)
train_rows <- sample(seq_len(nrow(mtcars)),
                     size = floor(0.8 * nrow(mtcars)))

train_data <- mtcars[train_rows, ]
test_data <- mtcars[-train_rows, ]

# Fit regression model on training data
case_model <- lm(mpg ~ wt + hp + disp + qsec,
                 data = train_data)

# Display model summary
summary(case_model)

# Predict on unseen test data
actual_mpg <- test_data$mpg
predicted_mpg <- predict(case_model, newdata = test_data)

# Calculate evaluation metrics
rmse <- sqrt(mean((actual_mpg - predicted_mpg)^2))
```

```
r_squared <- 1 -
  sum((actual_mpg - predicted_mpg)^2) /
  sum((actual_mpg - mean(actual_mpg))^2)

cat("Test RMSE:", rmse, "\n")
cat("Test R-squared:", r_squared, "\n")

# Compare actual and predicted values
results <- data.frame(
  Actual_MPG = actual_mpg,
  Predicted_MPG = as.numeric(predicted_mpg)
)

print(results)
```

Important Note: Install required packages before running. Ridge regression requires numeric model-matrix predictors.

Function Used / Meaning:

predict() – Generates predictions from the fitted model.

coef() – Extracts model coefficients.

cv.glmnet() – Fits a regularized model using cross-validation; alpha=0 specifies ridge regression.

set.seed() – Sets the random-number seed so results involving randomness can be reproduced.

model.matrix() – Creates the numeric model matrix used by the regression model.

vif() – Calculates Variance Inflation Factor to help detect multicollinearity.

cor() – Calculates correlations between numerical variables.

lm() – Fits the ordinary least squares regression model.

data() – Loads the mtcars dataset.

library() – Loads an installed R package.

Output:

1)10A

```
> # Load dataset
> data(mtcars)
> # Fit multiple linear regression model
> ols <- lm(mpg ~ wt + hp + disp + qsec, data = mtcars)
> # Correlation matrix
> cor(mtcars[, c("wt", "hp", "disp", "qsec")])
            wt         hp       disp       qsec
wt    1.0000000  0.6587479  0.8879799 -0.1747159
hp    0.6587479  1.0000000  0.7909486 -0.7082234
disp  0.8879799  0.7909486  1.0000000 -0.4336979
qsec -0.1747159 -0.7082234 -0.4336979  1.0000000
> # Variance Inflation Factor
> vif(ols)
      wt       hp     disp     qsec
6.916942 5.166758 7.985439 3.133119
```

2)10B

```
> # Prepare predictor matrix and target variable
> x <- model.matrix(mpg ~ wt + hp + disp + qsec,
+                   data = mtcars)[, -1]
> y <- mtcars$mpg
> # Ridge regression with cross-validation
> set.seed(123)
> cv_ridge <- cv.glmnet(x, y, alpha = 0)
> # Best regularization parameter
> cv_ridge$lambda.min
[1] 0.5648805
> # Ridge regression coefficients
> coef(cv_ridge, s = "lambda.min")
5 x 1 sparse Matrix of class "dgCMatrix"
              lambda.min
(Intercept) 29.470535240
wt          -3.161143854
hp          -0.019958095
disp        -0.007668484
qsec         0.307426003
> # Predictions for first five observations
> predict(cv_ridge, newx = x[1:5, , drop = FALSE],
+         s = "lambda.min")
                  lambda.min
Mazda RX4           22.82622
Mazda RX4 Wag       22.19229
Datsun 710          25.17358
Hornet 4 Drive      21.10996
Hornet Sportabout   17.57527
```

3)10C

```
> # Split data into training and testing sets
> set.seed(123)
> train_rows <- sample(seq_len(nrow(mtcars)),
+                      size = floor(0.8 * nrow(mtcars)))
> train_data <- mtcars[train_rows, ]
> test_data <- mtcars[-train_rows, ]
> # Fit regression model on training data
> case_model <- lm(mpg ~ wt + hp + disp + qsec,
+                  data = train_data)
> # Display model summary
> summary(case_model)

Call:
lm(formula = mpg ~ wt + hp + disp + qsec, data = train_data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.3522 -1.9808 -0.5129  0.9580  6.2570

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 21.95105    9.78063   2.244  0.03628 *
wt          -4.89667    1.48277  -3.302  0.00356 **
hp          -0.01003    0.01719  -0.583  0.56615
disp        -0.00043    0.01249  -0.034  0.97287
qsec         0.87032    0.53618   1.623  0.12021
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.699 on 20 degrees of freedom
Multiple R-squared:  0.8531,    Adjusted R-squared:  0.8237
F-statistic: 29.04 on 4 and 20 DF,  p-value: 4.456e-08

> # Predict on unseen test data
> actual_mpg <- test_data$mpg
> predicted_mpg <- predict(case_model, newdata = test_data)
> # Calculate evaluation metrics
> rmse <- sqrt(mean((actual_mpg - predicted_mpg)^2))
> r_squared <- 1 -
+   sum((actual_mpg - predicted_mpg)^2) /
+   sum((actual_mpg - mean(actual_mpg))^2)
> cat("Test RMSE:", rmse, "\n")
Test RMSE: 2.649541
> cat("Test R-squared:", r_squared, "\n")
Test R-squared: 0.4070891
```

```
> # Compare actual and predicted values
> results <- data.frame(
+   Actual_MPG = actual_mpg,
+   Predicted_MPG = as.numeric(predicted_mpg)
+ )
> print(results)
  Actual_MPG Predicted_MPG
1       21.0     21.513759
2       18.1     21.456436
3       16.4     15.240997
4       17.3     17.079930
5       10.4      8.546152
6       21.5     26.271210
7       19.2     16.034889
```

Conclusion: Multicollinearity was examined using correlations and VIF, and ridge regression was fitted with cross-validation. The practical demonstrated a complete regression workflow from inspection to validation.

Course Outcome: After completing this practical, the student will be able to identify multicollinearity, apply ridge regression with cross-validation and follow a complete regression analysis workflow.

Viva Questions:

6. What is multicollinearity?
7. What does VIF stand for?
8. What is ridge regression?
9. What does the penalty parameter control?
10. Why validate a model on real data?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]